Ideation Phase

Brainstorm & Idea Prioritization Template

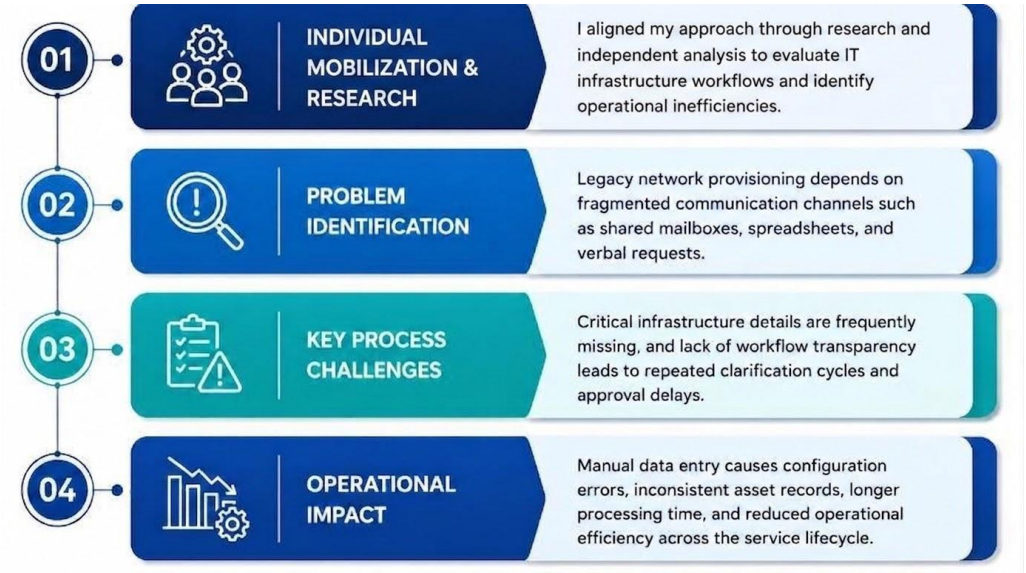
Date	25 July 2026
Name	Sakhamuri Ruthvick Sai
Project Name	Automated Network Request Management System
Maximum Marks	4 marks

Step-1: Individual Mobilization, Research, and Problem Identification

Individual Research & ApproachI independently evaluated corporate IT infrastructure inefficiencies by analyzing operational workflows managed by enterprise service teams, specifically focusing on how network provisioning deployments, physical asset modifications, and workspace relocation requests are handled.

Defined Problem StatementMy discovery phase revealed that the legacy framework for managing corporate network provisioning depends heavily on fragmented, decentralized communication pathways—such as unmonitored shared mailboxes, manual spreadsheet tracking, and informal verbal requests. This structural deficiency introduces severe operational vulnerabilities:

- **Deficient Metadata Collection:** Requesters frequently omit business-critical physical infrastructure variables—such as device context requirements, terminal extension layouts, or exact structural building coordinates—triggering prolonged backend clarification cycles.
- **Absence of Process Transparency:** Requests routinely stall indefinitely within administrative queues due to the lack of an automated workflow engine or dynamic managerial authorization routing.
- **Systemic Data Integrity Decay:** Manual transcription and data-entry practices introduce frequent clerical errors, resulting in mismatched hardware serial configurations, broken asset lifecycle dependencies, and flawed network configuration records.



Step-2: Taxonomic Categorization & Brainstorming Repository

Group A: Presentation Layer & Front-End Validation Protocols

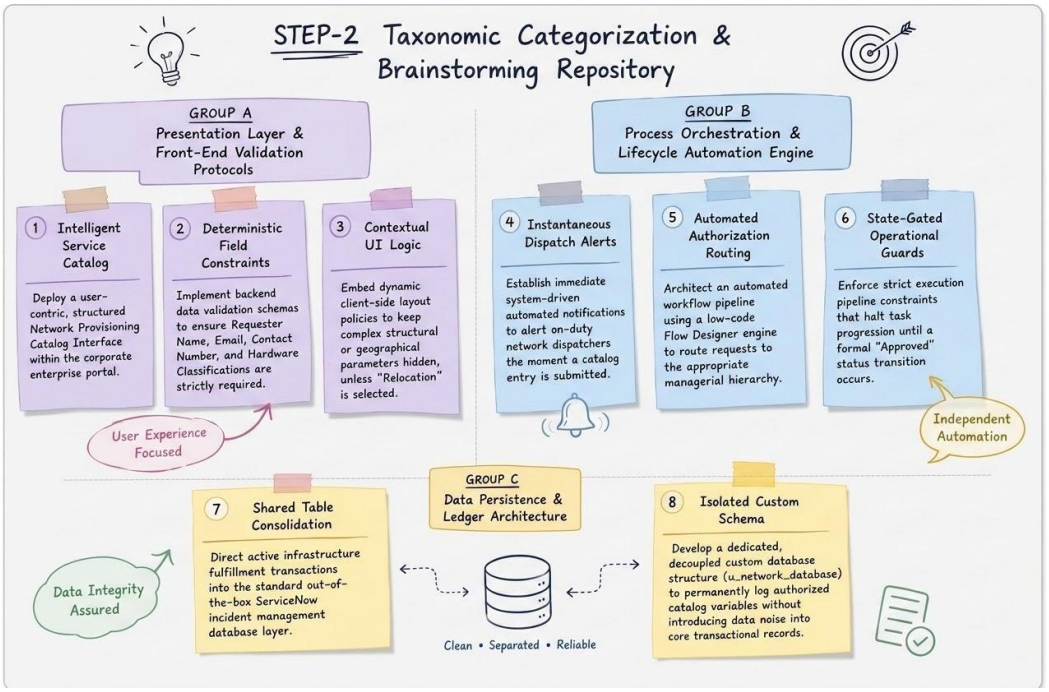
- System Component 1 (Intelligent Service Catalog): Deploy a user-centric, structured Network Provisioning Catalog Interface seamlessly integrated within the corporate enterprise service portal.
- System Component 2 (Deterministic Field Constraints): Implement rigorous backend data validation schemas to ensure vital identifying data and device metadata (specifically Requester Name, Email Address, Contact Number, and Hardware Classifications) are strictly required before compilation.
- System Component 3 (Contextual UI Logic): Embed dynamic client-side layout policies that optimize form visibility by keeping complex structural or geographical parameters hidden, unless the "New Connection vs. Relocation" control is toggled to "Relocation".

Group B: Process Orchestration & Lifecycle Automation Engine

- System Component 4 (Instantaneous Dispatch Alerts): Establish immediate system-driven automated notifications to alert on-duty network dispatchers the moment a catalog entry is successfully submitted.
- System Component 5 (Automated Authorization Routing): Architect an automated workflow management pipeline utilizing a low-code Flow Designer engine to dynamically capture inbound requests and route them along the appropriate managerial evaluation hierarchy.
- System Component 6 (State-Gated Operational Guards): Enforce strict execution pipeline constraints that programmatically halt task progression, preventing further fulfillment activities until a formal "Approved" status transition occurs.

Group C: Data Persistence & Ledger Architecture

- System Component 7 (Shared Table Consolidation): Direct active infrastructure fulfillment transactions straight into the standard out-of-the-box ServiceNow incident management database layer.
- System Component 8 (Isolated Custom Schema): Develop a dedicated, decoupled custom database structure (u_network_database) engineered specifically to permanently log authorized catalog variables without introducing data noise into core transactional records.



Step-3: Feasibility Analysis & Strategic Prioritization

To map out engineering feasibility against organizational value, all proposed architecture solutions were evaluated across a two-dimensional Effort vs. Impact Matrix. This categorization separates immediate structural requirements from legacy architectural anti-patterns.

Architectural Core Concept	Technical Complexity (Effort)	Definitive Operational Return (Impact)	Strategic Execution & Action
Dynamic Layout UI Policies	Low *(Client-side conditional visibility logic and view-state controls)*	High *(Optimizes user experience, hiding complexity unless contextually relevant)*	High Priority: Adopted to handle complex, conditional relocation parameters dynamically.

Architectural Core Concept	Technical Complexity (Effort)	Definitive Operational Return (Impact)	Strategic Execution & Action
Flow Designer Approval Orchestration	Medium *(State machine configurations and iterative multi-stage routing loops)*	High *(Completely eliminates manual tracking overhead and administrative delays)*	High Priority:Adopted as my primary automated process orchestration engine.
Isolated Custom Schema (u_network_database)	Medium *(Data dictionary definition and independent database table layout)*	High *(Maintains clean system architecture and isolates the network asset ledger)*	High Priority: Adopted to guarantee database health and clear relational separation.
Basic Email Broadcast Alerts	Low *(Standard platform-g enerated notification configurations)*	Low *(Fails to enforce accountability, create audit trails, or write structured data)*	Subordinated: Retained strictly as a auxiliary background activity log.